

6 MODELIRANJE I METODOLOGIJE RAZVOJA PROGRAMA

6.1 MODELIRANJE PROCESA

- 6.1.1 Dijagram strukture procesa
 - 6.1.1.1 Procesi transformacijske strukture
 - 6.1.1.2 Procesi transakcijske strukture
- 6.1.2 Detaljni opis procesa (dijagrami akcija)
 - 6.1.2.1 Proces
 - 6.1.2.2 Sekvencija
 - 6.1.2.3 Selekcija
 - 6.1.2.4 Iteracija
- 6.1.3 Pseudojezik i strukturirani prirodni jezik
 - 6.1.3.1 Stabla odlučivanja
 - 6.1.3.2 Tablice odlučivanja
 - 6.1.3.3
 - 6.1.3.4 Odlučivanje i dijagrami akcija

6.2 MODELIRANJE PODATAKA

- 6.2.1 Model objekti - veze - svojstva (ER model)
 - 6.2.1.1 Objekti
 - 6.2.1.2 Veze
 - 6.2.1.3 Svojstva
- 6.2.2 Relacijski model podataka I
- 6.2.3 RELACIJSKA ALGEBRA
 - 6.2.3.1 Projekcija
 - 6.2.3.2 Selekcija
 - 6.2.3.3 Spajanje

6.3 OPĆE DEFINICIJE OSNOVNIH SQL NAREDBI

- 6.3.1 OSNOVNE NAREDBE SQL- a (primjeri)
 - 6.3.1.1 CREATE
 - 6.3.1.2 INSERT
 - 6.3.1.3 DELETE
 - 6.3.1.4 UPDATE
 - 6.3.1.5 SELECT
- 6.3.2 Logički model baze podataka
 - 6.3.2.1 Logički modeli orijentirani entitetima
 - 6.3.2.2 Logički modeli orijentirani zapisima
 - 6.3.2.3 Fizički model podataka

6.4 Postupak razvoja programa

- 6.4.1 Svrha
- 6.4.2 Definicije pojmova
- 6.4.3 Odgovornosti i uloge
 - 6.4.3.1 Voditelj projekta
 - 6.4.3.2 Osiguranje kvalitete
- 6.4.4 Opis postupka
 - 6.4.4.1 Općenito

6.5 Faze razvoja programa s definiranim ulazima i izlazima

- 6.5.1.1 Testiranje (potvrda i ovjera programskog proizvoda)
- 6.5.1.2 Promjene u razvoju programa
- 6.5.2 Dokumentacija

6 MODELIRANJE I METODOLOGIJE RAZVOJA PROGRAMA

6.1 MODELIRANJE PROCESA

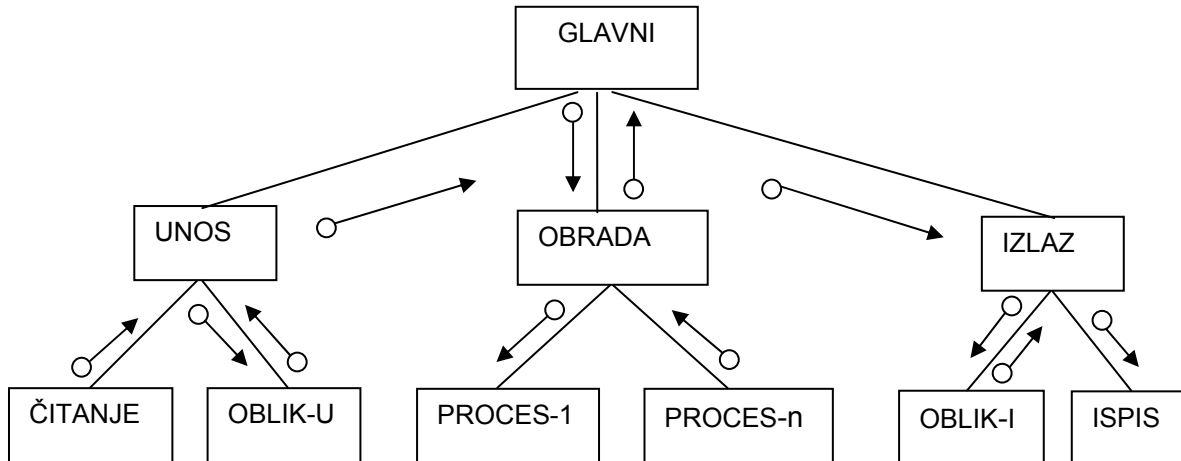
6.1.1 Dijagram strukture procesa

Primjena elementarnih kontrolnih struktura slijedi iz razlaganja (dekompozicije) ukupnog korisničkog posla (aplikacije). Poslove dekomponiramo na procese u hijerarhijskim razinama koje nazivamo "dijagrami strukture procesa".

Moduli iz dijagrama strukture procesa mogu biti pozivani iz više nadređenih modula. Općenito vrijedi da je pojedini podproces (modul) utoliko bolje oblikovan (identificiran) što se iz većeg broja (nadređenih) modula poziva.

6.1.1.1 Procesi transformacijske strukture

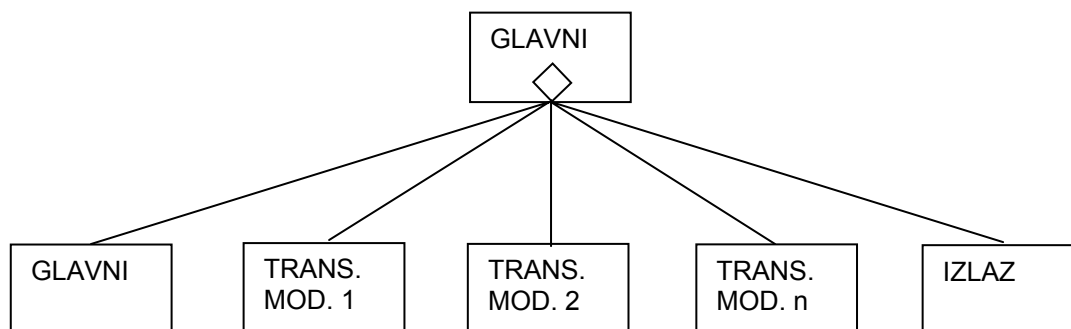
Za proces kažemo da je transformacijske strukture ako prihvata ulazne podatke, obavlja niz sukcesivnih operacija (transformacija) nad tim podacima, te generira rezultat procesa. Primjeri takvih obrada su serijske (batch) obrade.



6.1.1.2 Procesi transakcijske strukture

Za proces kažemo da je transakcijske strukture ako se njime izvodi veći broj različitih transakcija (prijenosa, ažuriranja) podataka. Izbor modula (tj. vrste transakcije) obavlja se na osnovi stanja nekog kontrolnog indikatora iz glavnog modula.

Programska rješenja oblikovana pomoću izbornika u "on-line" obradi predstavljaju najčešći način realizacije ovakve strukture.



6.1.2 Detaljni opis procesa (dijagrami akcija)

Procesi koji se navode u dijagramima toka podataka i dijagramima strukture procesa (te neformalno opisuju u popratnoj dokumentaciji), definiraju se detaljno i precizno jezikom dijagrama akcija. Te definicije procesa tvore osnovu za njihovu programsku realizaciju.

Jezik dijagrama akcija temelji se na strukturiranom pristupu definiranju procesa: tj. svaki sastavljeni proces može se opisati definiranjem sekvencije, selekcije i iteracije njegovih podprocesa.

6.1.2.1 Proces

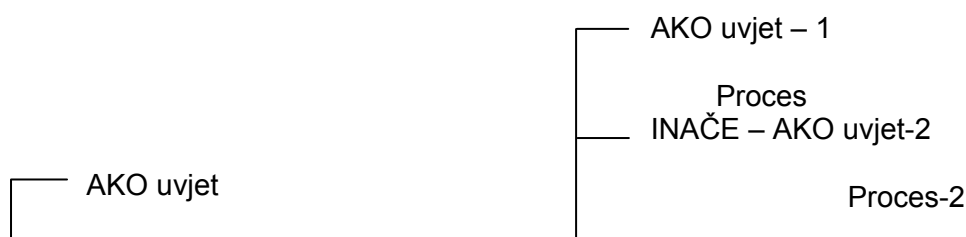
Proces može biti jednostavan ili sastavljen. Jednostavnim procesima nazovimo operacije poput PROČITAJ, NAPIŠI, IZRAČUNAJ... i nizove takvih operacija koji tvore (manje) zaokružene cjeline. Sastavljenima nazivamo procese koji su sastavljeni iz više jednostavni(ji)h procesa. Te procese nazivamo podprocesima (sastavljenog) procesa.

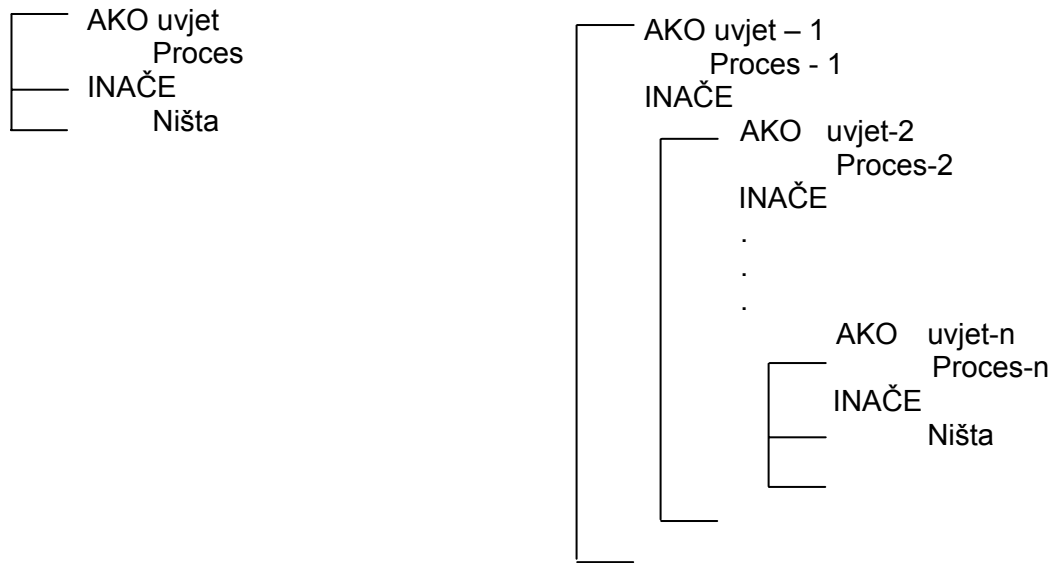
6.1.2.2 Sekvencija

Sekvencijom nazivamo niz (pod)procesa. Sastavljen proces - koji taj niz tvori - izvodi se tako da se redom izvode svi podprocesi iz niza.

6.1.2.3 Selekcija

Selekcija prikazuje način "grananja" procesa. Preciznije rečeno, simbol selekcije dijeli proces na dva disjunktna podprocesa. Proces koji sadrži selekciju izvodi se tako da se - u zavisnosti od (ne)ispunjenosti uvjeta selekcije - izvede točno jedan od dvaju njegovih podprocesa.





6.1.2.4 Iteracija

Iteracijom nazivamo višestruko izvođenje (tj. ponavljanje) istog procesa. Ponavljani proces se pritom redovito izvodi svaki put nad različitim (novim) podacima.

Da bi jednom započeto ponavljanje procesa stablo (završilo), izvođenje samog procesa treba na neki način utjecati na zadovoljenost toga uvjeta.

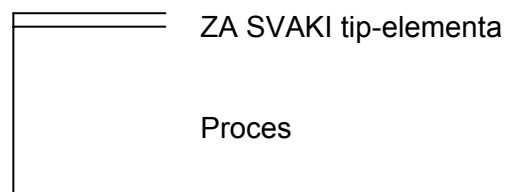
U programskim jezicima nalazimo više konstrukata koji realiziraju iteraciju. Na slici su dana tri osnovna oblika tih konstrukata:



(a)



(b)



(c)

Prema konstrukt (a), proces se ponavlja sve dok dani uvjet jest ispunjen. S druge strane, prema konstrukt (b), proces se ponavlja sve dok dani uvjet ne postane (tj. dok nije!) ispunjen. Dakle, u drugom slučaju, ispunjenjem uvjeta prekida se ponavljanje procesa.

Prema konstrukt (c), proces se izvodi za svaki od elemenata danoga tipa. To su najčešće svi podaci iz nekog spremišta (datoteke, relacije, tablice) podataka.

6.1.3 Pseudojezik i strukturirani prirodni jezik

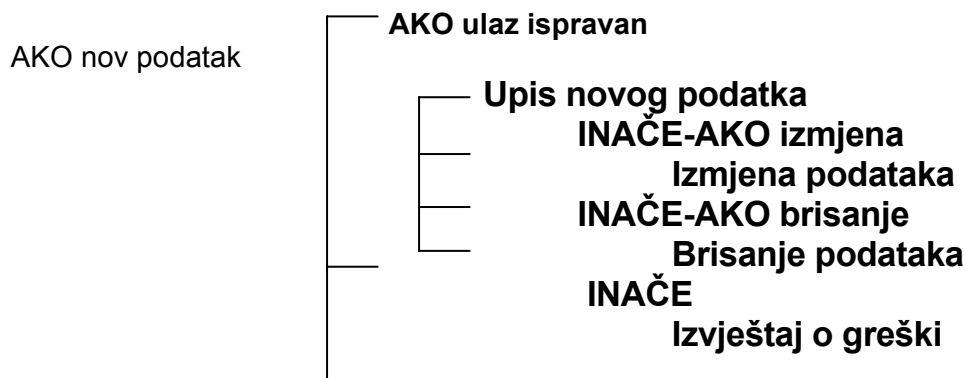
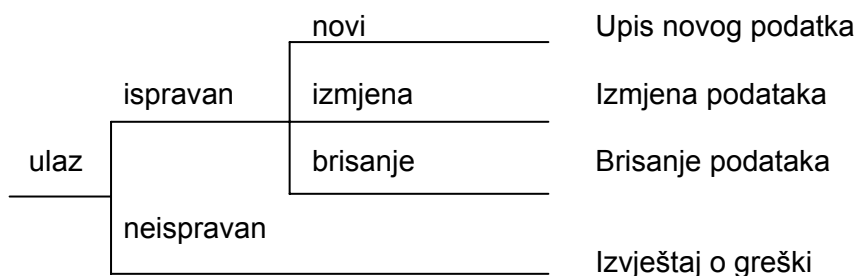
Kao jezici (sredstva) za opisivanje procesa često se navode strukturirani prirodni jezik i pseudojezik. Ti jezici nisu formalno definirani (a ni standardizirani), već se u pravilu određuju nizom preporuka o tome kakva treba biti struktura rečenica tih jezika.

Približno rečeno, strukturirani prirodni jezik je onaj fragment prirodnog jezika koji se daje direktno zapisati (prevesti) pomoću rečenica i konstrukata programskog jezika. S druge strane, pseudojezik je programski jezik oslobođen strogosti sintakse.

6.1.3.1 Stabla odlučivanja

Stablo odlučivanja prikazuje hijerarhijsko uređenje točki grananja odnosno uvjete izvođenja uvjetovanih (pod)procesa. U svakoj točki grananja dan je jedan uvjet, o čijoj (ne)ispunjenosti ovisi način (tj. smjer) nastavljanja procesa.

Stablo odlučivanja zorno prikazuje uvjete za pokretanje (izvođenje) nekog podprocesa, odnosno načina izračunavanja neke vrijednosti.



6.1.3.2 Tablice odlučivanja

6.1.3.3

Tablice odlučivanja pokazuju pod kojim se uvjetima aktiviraju (izvode) koji procesi. Po izražajnim mogućnostima, tablice odlučivanja ekvivalentne su stablima odlučivanja.

Međutim, postoje složeni procesi (ili, općenito: situacije) u kojima se javlja izrazito velik broj uvjeta i njima uvjetovanih (pod)procesa. Potpun opis takvih procesa (situacija) treba sadržavati sve moguće premise (tj. sve kombinacije ispunjenosti uvjeta) i njihove konsekvence (tj. pripadne uvjetovanje procese). Tablica odlučivanja je najbolje sredstvo za iscrpan i zoran prikaz takvih procesa, tj. za prikaz svih odnosa kombinacija uvjeta i njima uvjetovanih procesa.

AKO način plaćanja = A

AKO status < 10

AKO vrijednost > 1000

popust = 2

INAČE

popust = 4

INAČE-AKO $10 \leq \text{status} \leq 20$

AKO vrijednost < 1000

popust = 4

INAČE

popust = 6

INAČE-AKO status > 20

AKO vrijednost < 1000

popust = 6

INAČE

popust = 8

INAČE-AKO način plaćanja = B

AKO status < 10

AKO vrijednost < 1000

popust = 1

INAČE

popust = 2

INAČE

AKO vrijednost < 1000

popust = 2

INAČE

popust = 3

INAČE-AKO način plaćanja = C

AKO vrijednost < 1000

popust = 0

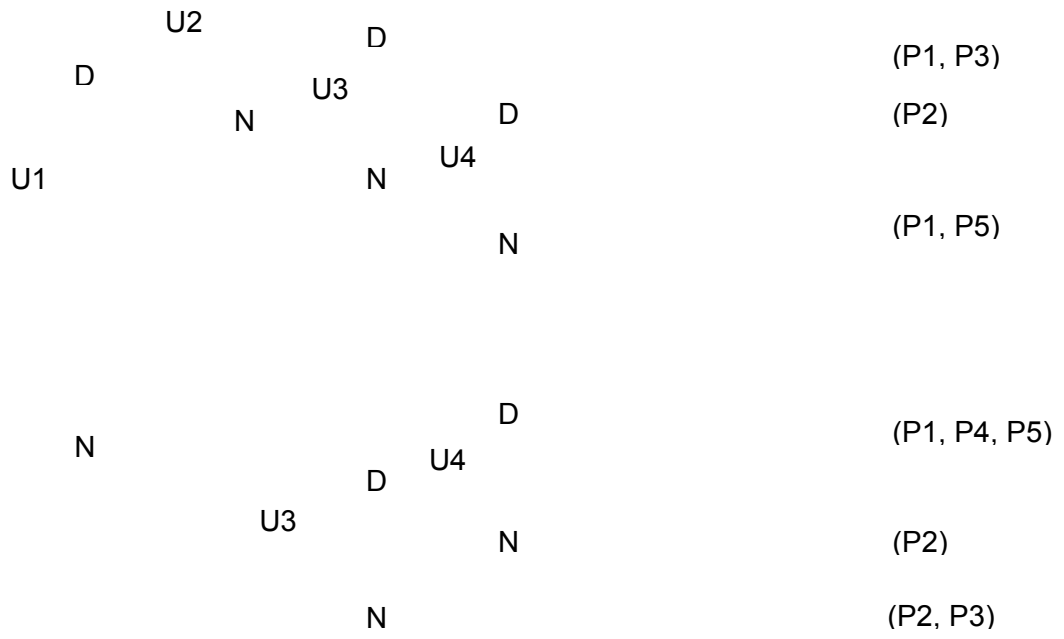
INAČE-AKO $1000 \leq \text{vrijednost} \leq 2000$

popust = 1

INAČE-AKO količina > 2000

popust=2

Opći oblik tablice odlučivanja:



```

AKO u1
  AKO u2
    (P1, P4)
  INAČE
    AKO u3
      (P1, P3)
    INAČE
      AKO u 4
        (P2)
      INAČE
        (P1, P5)

```

```

INAČE
  AKO u3
    AKO u4
      (P1, P4, P5)
    INAČE
      (P2)
  INAČE
    (P2, P3)

```

6.2 MODELIRANJE PODATAKA

6.2.1 Model objekti - veze - svojstva (ER model)

Model objekti - veze - svojstva, kao grafički jezik za predstavljanje struktura podataka, predložen je od CHENA 1976. godine. Taj model se obično naziva ER modelom podataka (Entity - Relationship) pa ćemo ga ovdje tako i nazivati.

Ishodište ER modela podataka možemo izreći približno slijedećim riječima:

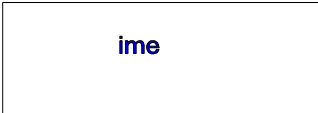
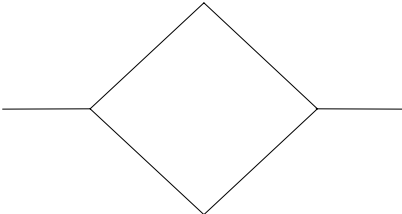
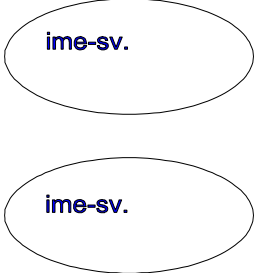
Podaci su znanja o objektima, vezama (među tim objektima) i svojstvima (objekata odnosno veza). Stoga je cilj jezika za modeliranje podataka omogućiti precizno i jednostavno predstavljanje znanja tih triju temeljnih pojmova.

Entitetom nazivamo nešto što se može jednoznačno imenovati.

Da bismo o sustavu mogli preciznije govoriti, entitete dijelimo (prema funkciji) na:

- objekte,
- veze (među objektima),
- svojstva (objekata i veza).

Osnovni simboli ER modela podataka prikazani su na slici:

Tip entiteta	Grafiki prikaz (simbol)
Objekt	
Veza	
Svojstvo: - deskriptor - identifikator	

6.2.1.1 Objekti

Okvirno rečeno, ulogu objekta u modelu najčešće imaju "čvrsti elementi" sustava. U pravilu su to primarni elementi sustava, o kojima pored svojstva - identifikatora postoji i niz svojstva - deskriptora. Takvi su naprimjer:

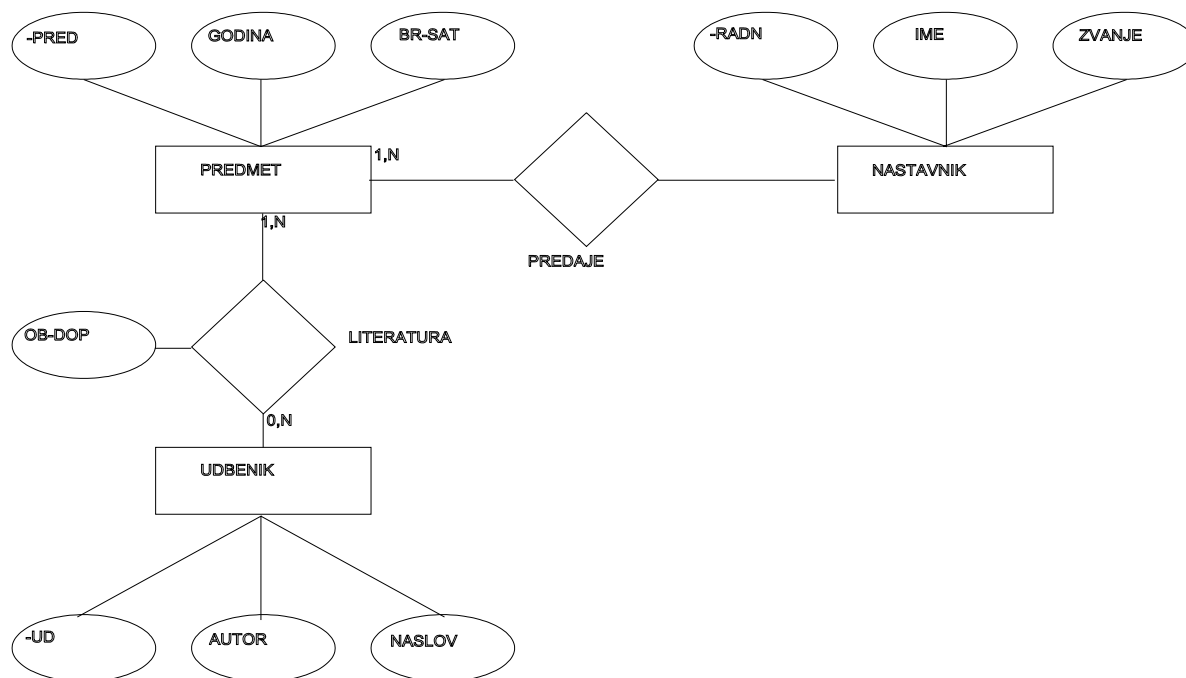
- entitet RADNIK (svojstva: ŠIFRA, IME, DATUM-ROĐENJA,...),
- entitet RAČUN (svojstva: BROJ, DATUM, IZNOS,...) i slični.

6.2.1.2 Veze

Simbolom veze iskazuju se veze među objektima sustava. Drugim riječima, simbol veze označava tip entiteta koji jest veza među danim tipovima objekata. Kao ilustrativni primjer takvog entiteta navedimo vezu PREDAJE između objekta PREDMET i NASTAVNIK. Poput objekata, veze mogu imati svojstva, kao što su datum uspostavljanja veze i slično.

6.2.1.3 Svojstva

Svojstvima - identifikatorima nazivamo ona svojstva čije vrijednosti omogućavaju jednoznačno identificiranje svake od pojava danog tipa objekta. Naprimjer, vrijednost (fiktivnog) svojstva Š-RADN (tj. šifra radnika) jednoznačno identificira jednog radnika, kao instancu tipa objekta RADNIK.



6.2.2 Relacijski model podataka I

Relacija, kao temeljni element relacijskog modela, ima dva aspekta: značenje i sadržaj. Značenje relacije naziva se intenzijom, a formalno se iskazuje shemom relacije. Sadržaj relacije naziva se ekstenzijom, a iskazuje se naslovljenom tabelom podataka.

Ilustrirajmo to primjerom. Neka relacija NARUDŽBA ima slijedeće (neformalno) značenje:

Kupci naručuju artikle u određenim količinama.

Značenje te relacije zapisuje se u relacijskom jeziku slijedećom shemom relacije:

NARUDŽBA (KUPAC, ARTIKL, KOLIČINA).

Schema relacije ne predstavlja podatke, već njihovu formu, a interpretacijom i značenje.

Sadržaj relacije NARUDŽBA tvori skup zapisa o pojedinačnim narudžbama. Primjer ekstenzije (tj. sadržaja) te relacije dan je na slici.

NARUDŽBA

KUPAC	ARTIKL	KOLIČINA
K1	A1	50
K2	A1	72
K2	A2	24
K3	A3	48

Schema relacije (tj. njeno značenje odnosno intenzija) ne mijenja se. S druge strane, sadržaj tabele (tj. ekstenzija relacije) mijenja se tijekom odvijanja poslovnog procesa. Naime, isporučitelj dobiva nove narudžbe, dok se stare (po izvršenju) brišu iz tabele/relacije.

Relacijska shema sastavljena je iz imena relacije i niza atributa. Skup mogućih vrijednosti atributa nazivamo domenom. U relacijskom modelu podataka, domene atributa su skupovi atomarnih vrijednosti. Neformalno rečeno, atomarnim vrijednostima nazivamo one vrijednosti koje nije moguće rastaviti, a da time ne bude uništeno njihovo značenje. Naprimjer, ako se ime kupca rastavi na skup znakova (slova), onda je ime izgubljeno.

Za relaciju čije sve domene sadrže samo atomarne vrijednosti kažemo da je normalizirana, odnosno da se nalazi u prvoj normalnoj formi. Iz rečenog slijedi da se svaka relacija iz relacijskog modela podataka nalazi u prvoj normalnoj formi. Drugim riječima, prva normalna forma relacije je kriterij prihvatljivosti oblika tabele u relacijskom modelu podataka.

Ekstenzija relacije je skup, čiji je sadržaj iskazan tabelom. Budući da se u skupu (po definiciji!) svaki element javlja samo jednom, tabela ne smije sadržavati dva identična retka. Nadalje, s obzirom da elementi skupa općenito nisu uređeni ("poredani"), redoslijed javljanja redaka u tabeli je nevažan. Drugim riječima, ako se ekstenzije dviju relacija razlikuju samo po redoslijedu javljanja redaka u tabeli, onda te dvije relacije imaju identične ekstenzije.

Analogno vrijedi i za redoslijed kolona tabele. Naime, svaka kolona - kao niz vrijednosti danog atributa - naslovljena je tim atributom. Stoga, izmjena redoslijeda kolona - zajedno sa pripadnim atributima - ne mijenja značenje relacije.

6.2.3 RELACIJSKA ALGEBRA

Neformalno rečeno, relacijskom bazom podataka možemo nazvati bazu podataka koju korisnik "vidi" kao skup normaliziranih tabela. (Dakle, relacijskim modelom podataka nije definiran sam način fizičke realizacije baze podataka).

Pored samog "tabelarnog izgleda", relacijski model karakterizira i skup operatora definiranih na skupu tabela - relacija. Ti operatori omogućuju da se, pored znanja (podataka) eksplicitno danih pojedinim relacijama, deduciraju (izračunaju) i ona znanja koja iz toga skupa relacija logički slijede.

Skup relacija zajedno sa skupom operatora (definiranih na tom skupu) tvori relacijsku algebru. Rezultat izvršenja operacije nad relacijama (iz određenog skupa relacija) je (nova) relacija. Kažemo, stoga, da je skup relacija zatvoren pod skupom operatora relacijske algebre.

Operatore relacijske algebre možemo podijeliti u dvije skupine: osnovne (primitivne) i izvedene. U osnovne spadaju selekcija, projekcija, kartezijski produkt, unija i razlika. Standardna tri izvedena operatora su uvjetno spajanje, prirodno spajanje i dijeljenje, koji su definirani na temelju osnovnih operatora. To ujedno ukazuje na mogućnost definiranja drugih (novih) operatora relacijske algebre. Navedena tri izvedena operatora uzimaju se kao standardni (i dovoljni), jer udovoljavaju praktičnim potrebama izvođenja operacija nad relacijama.

Značenje/učinak tih operatora izložiti ćemo na primjeru relacija KUPCI, ARTIKLI i NARUDŽBE. Sheme tih relacija neka glase:

KUPCI(Š-KUP, IME, SJEDIŠTE)
ARTIKLI(Š-ART, NAZIV, BOJA, CIJENA)
NARUDŽBE(Š-KUP, Š-ART, KOLIČINA)

Ključeve relacija označavamo podcrtavanjem atributa koji tvore ključ. Ilustrativni primjer ekstenzija tih relacija dan je na slici:

KUPCI

<u>Š-KUP</u>	IME	SJEDIŠTE
K1	Galeb	Zagreb
K2	Arena	Pula
K3	Mirna	Rovinj
K4	Badel	Zagreb
K5	Torpedo	Rijeka

ARTIKLI

<u>Š-ART</u>	NAZIV	BOJA	CIJENA
A1	olovka	crvena	3
A2	gumica	bijela	7
A3	penkala	plava	8
A4	penkala	crvena	9
A5	tinta	plava	5
A6	papir	bijela	1

NARUDŽBE

Š-KUP	Š-ART	KOLIČINA
K1	A1	100
K1	A2	200
K1	A3	100
K1	A4	300
K1	A5	400
K1	A6	100
K2	A1	200
K2	A2	200
K3	A2	100
K4	A2	100
K4	A4	200
K4	A5	300

6.2.3.1 Projekcija

Operacija projekcije generira relaciju koja sadrži samo odabrane kolone iz dane relacije. Zapis operacije projekcije je slijedećeg oblika:

PROJEKCIJA (ime-relacije)
[niz-atributa]

Niz atributa sadrži one attribute dane relacije čije pripadne kolone želimo da sadrži relacija generirana operacijom projekcije. Pritom, atributi ne moraju biti navedeni istim redom u kojem se javljaju u polaznoj relaciji. Naprimjer, cijene artikala možemo saznati operacijom

PROJEKCIJA (ARTIKLI)
[Š-ART,CIJENA]

Rezultat izvođenja te operacije je relacija CIJENE sa slike:

CIJENE

Š-ART	CIJENA
A1	3
A2	7
A3	8
A4	9
A5	5
A6	1

6.2.3.2 Selekcija

6.2.3.3 Spajanje

Spajanje relacija potrebno je za generiranje onih odgovora koji ne slijede iz jedne same relacije. Takav je, naprimjer, odgovor na pitanje "S kojih je sve lokacija (tj. SJEDIŠTA) naručen artikal A2".

Definicija operacije spajanja temelji se na operatoru kartezijskog produkta i operatoru selekcije.

Operator selekcije generira relaciju koja sadrži sve one n-torke iz dane relacije za koje se uvjet evaluira na vrijednost istina. Zapis operacije selekcije je slijedećeg oblika:

IZBOR (ime-relacije)
uvjet.

Prirodno spajanje je spajanje u kojem je uvjet spajanja jednakost vrijednosti istoimenih atributa i koje u generiranoj relaciji izostavlja po jednu iz parova identičnih kolona.

Zapis operacije uvjetnog spajanja je slijedećeg oblika:

R1 SPOJ R2
uvjet

Primjenom prirodnog spajanja možemo definirati operaciju koja nalazi sjedišta svih naručioca artikla A2 na slijedeći način:

PROJEKCIJA (IZBOR (KUPCI SPOJ NARUDŽBE))
[SJEDIŠTE] N.Š-ART = A2

Rezultat izvođenja te operacije jest relacija na slici:

ODGOVORI

SJEDIŠTE
Zagreb
Pula
Rovinj

6.3 OPĆE DEFINICIJE OSNOVNIH SQL NAREDBI

CREATE TABLE

Naredba CREATE TABLE kreira tablicu određene strukture (nazivi polja tablice s tipovima i dimenzijama polja tablice).

Sintaksa

```
CREATE TABLE tablica(polje1 tip&dim_polja1 [, polje2 tip&dim_polja 2]...);
```

INSERT

Definicija

Naredba INSERT dodaje nove zapise u određenu tablicu.

Sintaksa

```
INSERT INTO tablica  
VALUES (konstanta/varijabla1 [, konstanta/varijabla2]...);
```

DELETE

Definicija

Naredba DELETE briše zapise određene tablice.

Sintaksa

```
DELETE FROM tablica  
[WHERE uvjet];
```

UPDATE

Definicija

Naredba UPDATE mijenja podatke u određenoj tablici.

Sintaksa

```
UPDATE tablica  
SET polje_x = izraz1 [, polje_y = izraz2]...  
[WHERE uvjet];
```

SELECT

Definicija

Naredba SELECT izabire polja (stupce) i zapise (retke) određene tablice.

Sintaksa

```
SELECT { * | polje_x [, polje_y]...}  
FROM tablica  
[WHERE uvjet];
```

Primjeri za **tip i dimenziju polja**:

```
NUMBER(2)  
CHAR(14)
```

Primjeri za **konstantu**:

```
10  
'New York'
```

Primjeri za **uvjet**:

```
polje_x < polje_y  
polje_x = konstanta AND polje_y > varijabla
```

Primjeri za **izraz**:

```
polje_x + 100  
polje_x / polje_y * 100
```

6.3.1 OSNOVNE NAREDBE SQL- a (primjeri)

6.3.1.1 CREATE

Kreira tablicu sa željenom strukturom.

```
CREATE TABLE ODJEL(SIF_ODJELA NUMBER(2),  
                    NAZIV CHAR (14),  
                    MJESTO CHAR (13));
```

Na isti način kreiramo i *ZAPOSLENIK* tablicu (kao što je na slici).

6.3.1.2 INSERT

Umeće zapise u postojeću tablicu.

```
INSERT INTO ODJEL  
VALUES (10,'RACUNOVODSTVO','ZAGREB');
```

Na isti način možemo popuniti tablicu *ODJEL* do kraja kao i *ZAPOSLENIK* tablicu (kao što je na slici).

6.3.1.3 DELETE

Brisanje zapisa koji zadovoljavaju određeni uvjet.

```
DELETE FROM ODJEL
WHERE SIF_ODJELA = 40;
```

sadržaj tablice *ODJEL* nakon brisanja:

<u>SIF_ODJELA</u>	<u>NAZIV</u>	<u>MJESTO</u>
10	RACUNOVODSTVO	ZAGREB
20	ISTRAZIVANJE	VARAZDIN
30	PRODAJA	ZAGREB

6.3.1.4 UPDATE

Mijenjanje vrijednosti polja u onim zapisima koji zadovoljavaju postavljeni uvjet.

```
UPDATE ZAPOSLENIK
SET PLACA = PLACA + 100
WHERE POSAO = 'SLUZBENIK';
```

Izgled promijenjenih zapisa *ZAPOSLENIK* tablice (polja *IME, POSAO, PLACA*)

<u>IME</u>	<u>POS AO</u>	<u>PLACA</u>
VESNA	SLUZBENIK	900.00
DIJANA	SLUZBENIK	1,200.00
MILAN	SLUZBENIK	1,050.00
LUKA	SLUZBENIK	1,400.00

6.3.1.5 SELECT

Pomoću ove naredbe obavljamo upit nad nekom tablicom. Kao rezultat upita na ekranu se pojave selektirana polja (ona polja koja želimo prikazati) i zapisi tablice (oni zapisi koji zadovoljavaju određeni kriterij).

```
SELECT *
FROM ODJEL;
```

ispis na ekranu:

<u>SIF_ODJELA</u>	<u>NAZIV</u>	<u>MJESTO</u>
10	RACUNOVODSTVO	ZAGREB
20	ISTRAZIVANJE	VARAZDIN
30	PRODAJA	ZAGREB
40	PROIZVODNJA	SISAK

```
SELECT SIF_ODJELA, NAZIV
FROM ODJEL;
```

ispis na ekranu:

<u>SIF_ODJELA</u>	<u>NAZIV</u>
10	RACUNOVODSTVO
20	ISTRAZIVANJE
30	PRODAJA
40	PROIZVODNJA

```
SELECT *
FROM ODJEL
WHERE SIF_ODJELA = 30;
```

ispis na ekranu:

<u>SIF_ODJELA</u>	<u>NAZIV</u>	<u>MJESTO</u>
30	PRODAJA	ZAGREB

```
SELECT IME, POSAO, PLACA
FROM ZAPOSLENIK
WHERE POSAO = 'DIREKTOR'
AND PLACA > 2800;
```

ispis na ekranu:

<u>IME</u>	<u>POS AO</u>	<u>PLACA</u>
MARKO	DIREKTOR	2,975.00
DANIJEL	DIREKTOR	2,850.00

```
SELECT IME, POSAO, PLACA
FROM ZAPOSLENIK
WHERE POSAO = 'DIREKTOR'
OR PLACA > 2800;
```

ispis na ekranu:

<u>IME</u>	<u>POS AO</u>	<u>PLACA</u>
MARKO	DIREKTOR	2,975.00
DANIJEL	DIREKTOR	2,850.00
ŽELJKO	DIREKTOR	2,450.00
MIRKO	ANALITICAR	3,000.00
IVAN	PREDSJEDNIK	5,000.00
DRAGAN	ANALITICAR	3,000.00

ODJEL Table

SIF_ODJELA	NAZIV	MJESTO
10	RACUNOVODSTVO	ZAGREB
20	ISTRAZIVANJE	VARAZDIN
30	PRODAJA	ZAGREB
40	PROIZVODNJA	SISAK

ZAPOSLENIK Table

ZAPOSLENIK	IME	POSAO	MGR	DATUM	PLACA	RATA	SIF_ODJELA
7369	VESNA	SLUZBENIK	7902	17-DEC-80	800.00		20
7499	ALEN	PRODAVAC	7698	20-FEB-81	1,600.00	300.00	30
7521	DARKO	PRODAVAC	7698	22-FEB-81	1,250.00	500.00	30
7566	MARKO	DIREKTOR	7839	02-APR-81	2,975.00		20
7654	MARTIN	PRODAVAC	7698	28-SEP-81	1,250.00	1,400.00	30
7698	DANIJEL	DIREKTOR	7839	01-MAY-81	2,850.00		30
7782	ŽELJKO	DIREKTOR	7839	09-JUN-81	2,450.00		10
7788	MIRKO	ANALITICAR	7566	09-NOV-81	3,000.00		20
7839	IVAN	PREDSJEDNIK		17-NOV-81	5,000.00		10
7844	MILIVOJ	PRODAVAC	7698	08-SEP-81	1,500.00	0.00	30
7876	DIJANA	SLUZBENIK	7788	23-SEP-81	1,100.00		20
7900	MILAN	SLUZBENIK	7698	03-DEC-81	950.00		30
7902	DRAGAN	ANALITICAR	7566	03-DEC-81	3,000.00		20
7934	LUKA	SLUZBENIK	7782	23-JAN-82	1,300.00		10

6.3.2 Logički model baze podataka

6.3.2.1 Logički modeli orijentirani entitetima

Postoji preko trideset logičkih modela podataka orijentiranih entitetima. Među njima dominantnu ulogu i daleko najveću primjenu ima ER model podataka.

6.3.2.2 Logički modeli orijentirani zapisima

Pored logičke strukture podataka, logički modeli orijentirani zapisima daju i globalni prikaz načina implementacije baze podataka. Najpoznatiji modeli toga tipa jesu relacijski, mrežni i hijerarhijski.

6.3.2.3 Fizički model podataka

Za razliku od logičkih modela podataka, fizičkih modela ima relativno malo. Ti modeli podataka definiraju način fizičkog zapisivanja (i organizacije) podataka na samim nosiocima podataka (diskovima,...).

Problematika optimalne fizičke realizacije baze podataka spada u domenu rada specijalista za konkretan sustav (na kojem se baza realizira).

6.4 Postupak razvoja programa

6.4.1 Svrha

Svrha ovladavanja razvojem programa je:

- obavljati razvojne aktivnosti pod kontroliranim uvjetima, na organiziran i stručan način,
- razviti program u skladu s praksom i iskustvima programskog inženjeringa, čime se smanjuje potreba za procesima potvrde i ovjere što dovodi do smanjenje troškova i poboljšanja kvalitete razvijenog programa,
- osigurati da konačni proizvod u cijelosti mora zadovolji zahtjeve koji su unaprijed definirani od strane naručitelja (kupac).

6.4.2 Definicije pojmova

Pojmovi koje koristimo u razvoju programa su slijedeći:

Faza (phase) – definirani segment rada,

Potvrda (verification) – aktivnost procjene proizvoda promatrane faze radi osiguranja ispravnosti i konzistentnosti u odnosu na proizvode i standarde uzete kao ulaze za tu fazu

Ovjera (ili valjanost; eng: validation) – proces vrednovanja programa radi udovoljavanja dogovorenim zahtjevima

Razvoj (development) - sve aktivnosti životnog ciklusa koje obuhvaćaju potrebne analize, dizajn, kodiranje, integraciju, testiranje, instalaciju (Setup proceduru) i podršku za prihvatanje programskog proizvoda

Model životnog ciklusa (life cycle model) – okvir koji obuhvaća procese, aktivnosti i zadatke sadržane u razvoju, izvođenju i održavanju programskog proizvoda od trenutka njegove definicije do prestanka njegove upotrebe

Programski proizvod (software product) – skup kompjuterskih programa, postupaka i moguće pridružene dokumentacije i podataka

Programski inženjering (software engineering) – sustavan pristup razvoju, izvođenju, održavanju i povlačenju iz uporabe programa

Element programa (programski modul) – bilo koji utvrdivi dio programskog proizvoda

Projektni zadatak (Plan projekta) – dokument kojim se definira opseg posla i potrebni resursi

Odluka o otvaranju projekta – dokument kojim direktor dozvoljava realizacija projektnog zadatka

Planovi realizacije projekta (Plan aktivnosti projekta) – operativni planovi koje donosi voditelj projekta u cilju realizacije posla

6.4.3 Odgovornosti i uloge

6.4.3.1 Voditelj projekta

Voditelj projekta upravlja radom slijedećih aktivnosti: Razvoja SW, Održavanje SW, te je odgovoran za:

1. uspostavu jasne organizacijske strukture (projektni tim)
2. praćenje, izbor i prilagodbu metodologije rada,
3. osiguranje tehnološke i tehničko-materijalne podrške u radu,
4. organizaciju kontinuiranog školovanja,
5. sve aktivnosti planiranja i praćenja napredovanja planova na razvoju programa.

Voditelj projekta razvoja programa također je odgovoran za koordinaciju rada unutar projektnog tima te za pravovremeno izvješćivanje naručitelja/direktora o napredovanju posla i za razrješavanje mogućih problema unutar projektnog tima i rada na projektu.

6.4.3.2 Osiguranje kvalitete

Upravitelj kvalitete (QA manager) djeluje prema predviđenim planovima razvoja programskih proizvoda ili projekta.

Upravitelj kvalitete usklađuje planove s planovima internih provjera te organizira i provodi interne provjere kvalitete postupaka i programskog proizvoda u konačnici.

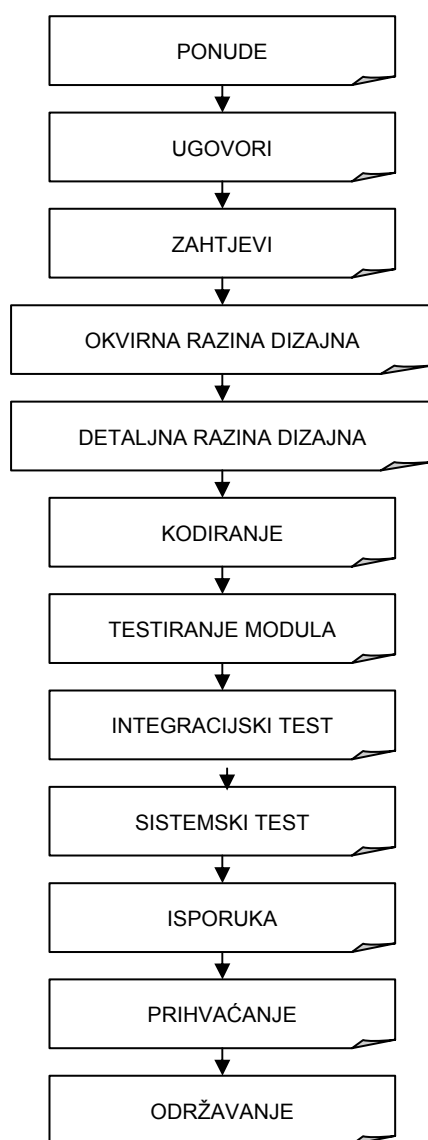
6.4.4 Opis postupka

6.4.4.1 Općenito

Razvoj programa organizira se u skladu s jednim ili kombinacijom različitih modela životnog ciklusa programa.

Model životnog ciklusa utvrđuje skup procesa i određuje način kada i kako će se oni primjenjivati. Procesi, aktivnosti i zadaci planiraju se i izvode u skladu s prirodom izabranog modela životnog ciklusa programa. Model životnog ciklusa može se prilagoditi tako da odgovara potrebama određenog projekta (slika 1).

Životni ciklus programskog proizvoda, uzevši u obzir i početne zahtjeve za rješavanjem problema, kao i problematiku ugovaranja poslova, može se grafički prikazati na način prikazan na slici.



Crtež 1.

Grafički prikaz životnog ciklusa programskog proizvoda

6.5 Faze razvoja programa s definiranim ulazima i izlazima

Iz izabranog modela životnog ciklusa programa proizlaze faze razvoja programa te se proces razvoja programa organizira u skladu s uočenim fazama.

Metodologija planiranja i razvoja programa uobičajeno se temelji na svjetskom standardu: IEEE Standard 1058.1-1987. Detalji o tom standardu mogu se pogledati u dokumentu "Software development plan", Visual basic Project Management Journal, 23 studeni 1997. Dokument se nalazi na Web adresi: www.vbpmj.com. pod nazivom sdp.doc.

Za rad u skladu s izabranim modelom životnog ciklusa programskog proizvoda odgovoran je voditelj projekta.

Osnovni ulazi i izlazi pojedinih faza životnog ciklusa programskog proizvoda navedeni su u nastavku u tablici i ne ovise o primjenjenom programskom jeziku.

FAZA	ULAZ	IZLAZ
Ponuda Javni natječaj	Zahtjevi klijenta	Odgovor klijenta na ponudu Prihvaćena ponuda
Ugovaranje	Prihvaćena ponuda	Kontrola ugovora Potpisani ugovor
Postavljanje zahtjeva	Ugovor Zahtjevi za promjenama Zahtjevi klijenta	Specifikacija zahtjeva Pregled podataka Plan projekta Rezultati analize zahtjeva Planovi testiranja Nacrt korisničkih priručnika
Dizajn	Specifikacija zahtjeva Zahtjevi za promjenama Plan projekta Rezultati analize zahtjeva	Specifikacija dizajna (programskih modula i baze) Specifikacija modula Nacrt korisničkih priručnika
Kod(iranje)	Specifikacija modula Plan projekta Standardi kodiranja(VB) Specifikacija dizajna Zahtjevi za promjenama	Kod Plan projekta Popis modula za testiranje Planovi testiranja i slučajevi

Testiranje (Integracija)	Kod (svi moduli) Podaci za testiranje Popis modula za testiranje Plan projekta	Testirani moduli (Aplikacija) Lista završenih testiranja (Prema slučajevima) Podaci o pogreškama Plan projekta
Test razvoja	Testirana aplikacija (super useri) Podaci za testiranje Ulazni podaci (šifarnici) Plan projekta	Release sustava (Verzija) Podaci o pogreškama Plan projekta
Test sustava (Produkcija – nakon instalacije)	Korisnička dokumentacija Release sustav Testiranja sustava i slučajevi Podaci za testiranje Plan projekta	Produkcija Plan projekta Priručnici
Isporuka	Testirani sustav (Produkcija) Korisnička dokumentacija Plan projekta	Isporučeni sustav Instalacija (Setup procedura) Plan obuke Materijali za obuku Ocjena treninga
Prihvaćanje	Isporučeni sustav Test prihvaćanja Plan projekta	Potpisano prihvaćanje od klijenta Plan projekta Rezultati testa prihvaćanja, Podaci Izveštaj o završetku i rezultatima projekta
Održavanje	Ugovor o održavanju Prihvaćeni sustav Zahtjevi za promjenama Izveštaji o pogreškama	Plan održavanja Podaci o promjenama Ažurirani isporučeni sustav

6.5.1.1 Testiranje (potvrda i ovjera programskog proizvoda)

Testiranje (ispitivanje) programskog proizvoda primjereno je složenosti razvijanog programa i rizika koji taj razvoj donosi, te svakoj fazi modela životnog ciklusa razvoja programa.

Proces potvrde i ovjere provodi se sukladno s utvrđenim fazama životnog ciklusa programa i redovitom provedbom dobivamo uvid u to:

- da li se dobro provodi razvoj programa (potvrda ili verifikacija postupka) i
- da li je programski proizvod kvalitetan (ovjera ili validacija proizvoda).

Aktivnosti koje se obavljaju a ovise o važnosti proizvoda i usvojenim planovima mogu biti:

- pregled svih ključnih elemenata i izlaza (npr. planovi, specifikacije analize ili detaljnog dizajna – projektna dokumentacija itd.),
- provjera sljedivosti korisničkih zahtjeva sa zahtjevima na program (poslovni zahtjevi),
- provjera sljedivosti zahtjeva na program s elementima analize i dizajna (informacijski zahtjevi),
- provjera formalnih dokaza i upotrebljenih metoda i algoritama (komponente, metodologija razvoja programa u VB okruženju, projektiranju relacijskih baza podataka, itd.).

6.5.1.2 Promjene u razvoju programa

Svaki razvojni model možemo podijeliti u dvije cjeline, a to su **logička izgradnja sustava** i **fizička izgradnja sustava**.

Budući da promjene zamisli (ideja) ili ulaznih specifikacija kod razvoja programa uzrokuju promjene projektnog zadatka i odgovarajućih planova, iskustveno se zna da takve promjene uzrokuju često neplanirane troškove i kašnjenja, kako za naručitelja a tako i za proizvođača. Stoga će proizvođač kroz ugovorne odnose s naručiteljem predvidjeti uvjete pod kojima će se promjene zahtijevati i postupke nadzora nad izmjenama u razvoju programa.

Kod logičke izgradnje sustava promjene u zamisli su prirodne i dozvoljene s ciljem izrade kvalitetne specifikacije. Kod fizičke izgradnje sustava promjene valja izbjegavati, a u slučaju da dolaze od korisnika smatrat će se proširenjem projektnog zadatka te će se regulirati dodacima ugovora.

Pošto se izmjene mogu dogoditi u bilo koje vrijeme životnog ciklusa proizvoda, one će se provesti slijedećim redoslijedom:

- a) dokumentirati,
- b) vrednovati posljedice koje donose,
- c) prihvatiti ili odbaciti,
- d) u slučaju prihvatanja primijeniti i potvrditi.

6.5.2 Dokumentacija

Razvoj programa zahtijeva slijedeću dokumentaciju i zapise:

- Projektni zadatak (Projektni zahtjev)
- Plan realizacije projekta (planovi)
- Odluka o pokretanju razvoja programa
- Primopredajni zapisnik.